# Бие даалт №2

## Графын хайлт, мөрийн хайлт ба хомхойлох алгоритм

Алгоритмын зохиомж ба шинжилгээ (F.CSM301)

А төвшин · Python, Java

Оюутан: Б. Билгүүн (**B190910014**)

Багш: Д. Батмөнх

ШУТИС, МХТС · Улаанбаатар, 2026

# Агуулга


**1 Ерөнхий мэдээлэл** **3**

1.1 Ажлын зорилго . . . 3

1.2 Файлын бүтэц . . . 3

1.3 Тестийн ерөнхий үр дүн . . . 3

1.4 Хэмжилтийн орчин ба арга . . . 4

**2 Бодлого 1. Find the Index of the First Occurrence in a String (LeetCode 28)** **4**

2.1 Зорилго . . . 4

2.2 Алгоритмын санаа . . . 4

2.3 Хэрэгжүүлэлт . . . 5

2.4 Тестийн үр дүн . . . 5

2.5 Хугацаа . . . 5

2.6 Санах ой . . . 5

**3 Бодлого 2. Flood Fill (LeetCode 733)** **5**

3.1 Зорилго . . . 5

3.2 Алгоритмын санаа . . . 6

3.3 Хэрэгжүүлэлт . . . 6

3.4 Тестийн үр дүн . . . 6

3.5 Хугацаа . . . 6

3.6 Санах ой . . . 6

**4 Бодлого 3. Assign Cookies (LeetCode 455)** **7**

4.1 Зорилго . . . 7

4.2 Алгоритмын санаа . . . 7

4.3 Хэрэгжүүлэлт . . . 7

4.4 Тестийн үр дүн . . . 7

4.5 Хугацаа ба санах ой . . . 7

**5 Бодлого 4. Jump Game II (LeetCode 45)** **8**

5.1 Зорилго . . . 8

5.2 Алгоритмын санаа (B төвшин): динамик программчлал . . . 8

5.3 Сайжруулсан шийдэл (A төвшин): хомхойлох арга = далд гүехэн хайлт . . . 8

5.4 Хэрэгжүүлэлт ба тестийн үр дүн . . . 9

5.5 Харьцуулалт . . . 9

**6 Бодлого 5. Open the Lock (LeetCode 752)** **9**

6.1 Зорилго . . . 9

6.2 Алгоритмын санаа (B төвшин): гүехэн хайлт . . . 9

6.3 Сайжруулсан шийдэл (A төвшин): хоёр талын гүехэн хайлт . . . 10

6.4 Хэрэгжүүлэлт ба тестийн үр дүн . . . 10

6.5 Харьцуулалт . . . 10

**7 Дүгнэлт** **11**

**A Java хэрэгжүүлэлт** **12**

A.1 1. strStr (`StrStr.java`) . . . . . 12

A.2 2. Flood Fill (`FloodFill.java`) . . . . . 12

A.3 3. Assign Cookies (`AssignCookies.java`) . . . . . 13

A.4 4. Jump Game II — анхны (`JumpGame.java`) . . . . . 13

A.5 4. Jump Game II — сайжруулсан (`JumpGameImproved.java`) . . . . . 14

A.6 5. Open the Lock — анхны (`OpenLock.java`) . . . . . 14

A.7 5. Open the Lock — сайжруулсан (`OpenLockImproved.java`) . . . . . 15

# 1 Ерөнхий мэдээлэл

## 1.1 Ажлын зорилго

Графын гүн ба гүехэн хайлт, тэмдэгт мөрийн хайлт болон хомхойлох алгоритмын зарчмыг хэрэгжүүлэх. A төвшний шаардлагын дагуу таван бодлогыг бүгдийг **Python** болон **Java** дээр бодож, B төвшний хоёр бодлогын гүйцэтгэлийг хомхойлох арга болон гүехэн хайлтаар **сайжруулав**. Бодлого бүрийн хугацаа, санах ойн зарцуулалтыг онолоор тооцоолж, бодит хэмжилтээр шалгав.

| № | Бодлого | Төвшин | Арга |
|---|---|---|---|
| 1 | Find the Index of the First Occurrence in a String (LeetCode 28) | C | KMP |
| 2 | Flood Fill (LeetCode 733) | C | гүехэн хайлт (BFS) |
| 3 | Assign Cookies (LeetCode 455) | C | хомхойлох |
| 4 | Jump Game II (LeetCode 45) | B → A | ДП → хомхойлох |
| 5 | Open the Lock (LeetCode 752) | B → A | BFS → хоёр талын BFS |

## 1.2 Файлын бүтэц

```
prj2/B190910014/
├── python/src/          p1_str_str.py ... p5_open_lock_improved.py
├── python/tests/        test_problems.py (pytest)
├── python/bench.py      хэмжилт
├── java/src/main/...    StrStr.java ... OpenLockImproved.java, Bench.java
├── java/src/test/...    ProblemsTest.java (JUnit 5)
├── report/              prj2_report.tex, prj2_report.pdf, make_tables.py, build.sh
└── env_info.sh
```

LeetCode-д илгээхдээ Python-д `class Solution`-ийг шууд, Java-д классын нэрийг `Solution` болгож (`package` мөрийг хасаж) илгээнэ.

## 1.3 Тестийн ерөнхий үр дүн

Бодлого бүрийг LeetCode-ийн жишээ болон **өөр, энгийн аргатай тулгасан санамсаргүй тохиолдлоор** шалгав. Анхны ба сайжруулсан хувилбарыг мөн хооронд нь тулгав.

| Бодлого | Шалгасан зүйл |
|---|---|
| 1 | 3000 санамсаргүй мөр: KMP ба энгийн арга нь хариугаараа (зөвхөн тест дээр) `str.find` / `indexOf`-той таарсан |
| 2 | 300 санамсаргүй зураг: тогтмол цэг хүртэл давтах (Python) ба дахин дуудах DFS (Java) лавлах хариутай |
| 3 | 500 санамсаргүй тохиолдол: хоёр талт графын хамгийн их хослол (Кун)-той |
| 4 | 500 санамсаргүй массив: ДП, хомхойлох арга ба BFS ижил; n = 300, nums[i] = 300 үед ДП яг $n(n-1)/2$ алхам |
| 5 | 300 санамсаргүй тохиолдол (0–400 түгжээ): BFS ба хоёр талын BFS ижил хариутай |

| Хэл | Команд | Үр дүн |
|---|---|---|
| Python | `cd python && .venv/bin/pytest -v` | 28 passed |
| Java | `cd java && mvn test` | Tests run: 5, Failures: 0 |

Санамсаргүй тулгалт нь **алдаа илрүүлсэн**: анхны BFS (5-р бодлого) зорилгыг түгжих хослолоос өмнө шалгадаг байсан тул зорилго өөрөө түгжих хослол үед $-1$-ийн оронд хариу буцааж байв. Шалгах дарааллыг сольж засаад, энэ тохиолдлыг тусгай тест болгон нэмэв.

### 1.4 Хэмжилтийн орчин ба арга

WSL2 Ubuntu 22.04, Intel Core i5-1334U (12 логик цөм, L2 7.5 MiB, L3 12 MiB), 7.6 GiB RAM, Python 3.10.12, OpenJDK 17.0.20.1 (`env_info.sh`).

- **Хугацаа.** Нэг дуудалт 20 ms-ээс богино бол багцаар давтаж, нэг дуудалтын дунджийг гаргав. Python-д 3, Java-д 5 багцын медиан. Java-д эхлээд 3 удаа дуудаж JIT-ийг халаана.
- **Санах ой.** Python: `tracemalloc`-ийн **оргил** утга. Java: `ThreadMXBean`-ээр нэг дуудалтад **нийт хуваарилсан** санах ой. Оролтыг аль алинд нь оролцуулаагүй.

Хэмжилтийг дахин хийж тайланг шинэчлэх дараалал:

```
cd python && python3 bench.py                              # -> out/results_python.tsv
cd java && java -Xss64m -cp build com.prj2.Bench           # -> out/results_java.tsv
cd report && python3 make_tables.py && ./build.sh          # хүснэгт -> .tex -> PDF
```

## 2 Бодлого 1. Find the Index of the First Occurrence in a String (LeetCode 28)

### 2.1 Зорилго

haystack мөрөнд needle мөр анх хаана орсныг (0-ээс эхэлсэн байрлал) олно. Ороогүй бол $-1$. Хэлний бэлэн хайлтын функц (`str.find`, `in`, `indexOf`) ашиглахгүй.

### 2.2 Алгоритмын санаа

**Энгийн арга** байрлал бүрээс needle-ийг тэмдэгт тэмдэгтээр харьцуулна. Тааралгүй бол дараагийн байрлалаас дахин эхэлнэ. Хамгийн муу үед (жишээ нь haystack = «aaa…ab», needle = «aa…ab») $O(n \cdot m)$.

**Кнут–Моррис–Праттын (KMP) арга** нь тааралгүй болоход аль хэдийн таарсан хэсгийг дахин шалгахгүй. Үүний тулд needle-ийн угтвар бүрийн **хамгийн урт жинхэнэ угтвар = дагавар**-ын урт fail[i]-г урьдчилан тооцоолно. Жишээ нь needle = «abab» бол fail = [0, 0, 1, 2]. haystack-ийг нэг удаа гүйхдээ k тэмдэгт таарсан байхад тааралгүй болбол k-г fail[k − 1] болгож ухраана, харин haystack-ийн заагчийг ухраахгүй.

```
def strStr(haystack, needle):
    m = len(needle)
    fail, k = [0] * m, 0
    for i in range(1, m):                       # угтвар функц
        while k > 0 and needle[i] != needle[k]:
            k = fail[k - 1]
        if needle[i] == needle[k]:
            k += 1
        fail[i] = k
    k = 0
    for i, ch in enumerate(haystack):           # хайлт
        while k > 0 and ch != needle[k]:
            k = fail[k - 1]
        if ch == needle[k]:
            k += 1
```

```
            if k == m:
                return i - m + 1
    return -1
```

### 2.3 Хэрэгжүүлэлт

- **Python** (`p1_str_str.py`): `Solution` (KMP), `SolutionNaive` (харьцуулах энгийн арга).
- **Java** (`StrStr.java`): `strStr` (KMP), `strStrNaive`. Тэмдэгтийг `charAt`-аар уншиж, шинэ мөр үүсгэхгүй.

### 2.4 Тестийн үр дүн

LeetCode-ийн жишээ («sadbutsad», «sad» $\to$ 0; «leetcode», «leeto» $\to -1$) болон «mississippi», «issip» $\to$ 4, «aaaaab», «aab» $\to$ 3 зэрэг fail функцийг шалгах тохиолдлууд хоёр хувилбарт давсан. «ab» цагаан толгойтой 3000 санамсаргүй хос дээр хариу ижил.

### 2.5 Хугацаа

**KMP: $\Theta(n + m)$.** Хайлтын давталтад k нь алхам бүрд хамгийн ихдээ 1-ээр өснө (нийт $\le$ n удаа), while давталт k-г дор хаяж 1-ээр бууруулна. $k \ge 0$ тул нийт бууралт нийт өсөлтөөс хэтрэхгүй, өөрөөр хэлбэл while нийтдээ $\le$ n удаа ажиллана (хорогдуулсан шинжилгээ). fail-ийг байгуулах нь үүнтэй адил $\Theta(m)$. **Энгийн арга** хамгийн муу үед $(n - m + 1) \cdot m$ харьцуулалт хийнэ.

| | | Python, ms | | Java, ms | | Санах ой, KB | |
|---|---|---|---|---|---|---|---|
| n | Оролт | KMP | энгийн | KMP | энгийн | Python KMP | Java KMP |
| 1 000 | муу | 0.334 | 9.779 | 0.309 | 0.363 | 1.0 | 0.4 |
| 10 000 | муу | 2.779 | 133.6 | 0.0646 | 1.432 | 1.0 | 0.4 |
| 100 000 | муу | 38.7 | 1 497 | 1.535 | 10.2 | 1.0 | 0.4 |
| 1 000 | санамсаргүй | 0.172 | 0.184 | 0.0562 | 0.0392 | 1.0 | 0.4 |
| 10 000 | санамсаргүй | 1.933 | 2.505 | 0.167 | 0.0728 | 1.0 | 0.4 |
| 100 000 | санамсаргүй | 66.5 | 29.0 | 1.970 | 2.011 | 1.0 | 0.4 |

**Дүгнэлт.** Муу оролт дээр $n = 10^5$, $m = 100$ үед KMP нь Python-д **39 дахин**, Java-д **6.6 дахин** хурдан. Харин санамсаргүй текстэд энгийн аргын харьцуулалт ихэнхдээ эхний тэмдэгт дээр тасардаг тул дунджаар $O(n)$ бөгөөд нэмэлт зардалгүйн улмаас Python-д KMP-ээс хурдан байна. Өөрөөр хэлбэл KMP-ийн давуу тал нь **хамгийн муу тохиолдлын баталгаа**. Java-гийн $n = 10^3$ мөрийн хугацаа JIT бүрэн халаагүй үеийн эхний хэмжилт тул бусдаасаа их байна.

### 2.6 Санах ой

KMP нь fail массивт $\Theta(m)$ ($m = 100$ үед Java-д 0.4 KB), энгийн арга $\Theta(1)$ нэмэлт санах ой хэрэглэнэ. Аль аль нь haystack-ийг хуулахгүй.

## 3 Бодлого 2. Flood Fill (LeetCode 733)

### 3.1 Зорилго

$R \times C$ зураг (sr, sc) нүднээс эхлэн, ижил өнгөтэй, дээш доош зүүн баруун талаараа холбогдсон бүх нүдийг шинэ өнгөөр будна.

### 3.2 Алгоритмын санаа

Нүдийг графын орой, ижил өнгөтэй хөрш нүдийг ирмэг гэж үзвэл бодлого нь эхлэх оройгоос хүрэх бүх оройг олох **графын хайлт** болно. Гүехэн хайлт (BFS) хийж, нүдийг дараалалд **оруулах үедээ** будна. Ингэснээр будагдсан нүд хуучин өнгөтэй тэнцэхээ больж, дахин дараалалд орохгүй (тусдаа visited массив хэрэггүй). Шинэ өнгө хуучинтайгаа ижил бол шууд буцаана. Эс бөгөөс будсан нүд хуучин өнгөтэй хэвээр үлдэж, хайлт зогсохгүй.

```
def floodFill(image, sr, sc, color):
    old = image[sr][sc]
    if old == color:
        return image
    image[sr][sc] = color
    queue = deque([(sr, sc)])
    while queue:
        r, c = queue.popleft()
        for nr, nc in ((r - 1, c), (r + 1, c), (r, c - 1), (r, c + 1)):
            if 0 <= nr < rows and 0 <= nc < cols and image[nr][nc] == old:
                image[nr][nc] = color
                queue.append((nr, nc))
    return image
```

### 3.3 Хэрэгжүүлэлт

- **Python** (`p2_flood_fill.py`): `Solution` (BFS, `deque`) ба `SolutionDfs` (тодорхой стектэй DFS). Дахин дуудах DFS нь 300 × 300 зурагт Python-ы стекийн хязгаарыг ($\approx 1000$) давдаг тул стек ашиглав.
- **Java** (`FloodFill.java`): дараалыг R·C урттай `int[]` массиваар, нүдийг r·C + c гэж нэг тоогоор кодолно (объект үүсгэхгүй).

### 3.4 Тестийн үр дүн

LeetCode-ийн хоёр жишээ болон 1–7 хэмжээтэй, 3 өнгөтэй 300 санамсаргүй зураг дээр BFS ба DFS лавлах хариутай таарсан. 300 × 300 зурагт DFS стекийн алдаагүй ажилласан.

### 3.5 Хугацаа

Нүд бүр дараалалд хамгийн ихдээ нэг удаа орж, 4 хөршөө шалгана: $\Theta(\mathbf{R{\cdot}C})$ (хамгийн муу үед бүх нүд ижил өнгөтэй). Доорх хэмжилтэд бүх нүд ижил өнгөтэй (бүгдийг будна).

| Зураг | Нүд | Python, ms | Java, ms | Python нс/нүд | Java нс/нүд | Python оргил, KB | Java хуваарилсан, KB |
|---|---|---|---|---|---|---|---|
| 50×50 | 2 500 | 7.973 | 0.888 | 3189 | 355.3 | 0.2 | 9.8 |
| 200×200 | 40 000 | 102.0 | 2.250 | 2551 | 56.3 | 0.2 | 156.3 |
| 1000×1000 | 1 000 000 | 1 547 | 53.2 | 1547 | 53.2 | 41.5 | 3 906 |

Нэг нүдэнд ногдох хугацаа Python-д 1 500–3 200 нс, Java-д 50–350 нс орчим. Зураг томрох тусам энэ утга тогтворжиж (жижиг зурагт тогтмол зардал, Java-д JIT нөлөөлнө), хугацаа нүдний тоотой шугаман өсөж байна.

### 3.6 Санах ой

Дараалал нэгэн зэрэг BFS-ийн хамгийн ихдээ хоёр төвшнийг хадгална. Хамгийн муу үед $\mathbf{O(R{\cdot}C)}$. Java хувилбар R·C урттай массивыг урьдчилан хуваарилдаг (1000 × 1000: 3.9 MB = $4{\cdot}10^6$ байт).

Python-ы `deque` зөвхөн тухайн үеийн төвшнийг хадгална. Бүтэн зурагт эхлэх булангаас тархахад төвшин нь диагональ бөгөөд O(R + C) хэмжээтэй тул оргил санах ой бага. Мөн Python-д өмнө чөлөөлөгдсөн tuple-ийг дахин ашигладаг (free list) тул `tracemalloc` жижиг утга харуулж байна.

# 4 Бодлого 3. Assign Cookies (LeetCode 455)

## 4.1 Зорилго

Хүүхэд бүр $g_i$ шуналтай, жигнэмэг бүр $s_j$ хэмжээтэй. $s_j \geq g_i$ бол хүүхэд сэтгэл ханамжтай. Хүүхэд бүрд хамгийн ихдээ нэг жигнэмэг өгөхөд сэтгэл ханамжтай хүүхдийн тоог хамгийн их болгоно.

## 4.2 Алгоритмын санаа

Хоёр массивыг эрэмбэлнэ. Жигнэмгийг жижгээс нь эхлэн авч, хангаж чадах бол одоогийн (шунал нь хамгийн бага) хүүхдэд өгнө. Эс бөгөөс энэ жигнэмгийг хэнд ч өгөх боломжгүй тул хаяна.

```
def findContentChildren(g, s):
    g.sort()
    s.sort()
    child = 0
    for cookie in s:
        if child < len(g) and cookie >= g[child]:
            child += 1
    return child
```

**Хомхойлох сонголт зөв (солих арга).** Хамгийн бага шуналтай $g_1$-ийг хангах хамгийн жижиг жигнэмэг $s_a$ гэе. Оновчтой хуваарилалт O-д $g_1$ нь $s_b$-г авсан бол ($b \neq a$) $s_a$ ба $s_b$-г сольж болно: $s_a \leq s_b$ тул $s_a$-г авч байсан хүүхэд $s_b$-ээр ч хангагдана. $g_1$ хүүхэд O-д юу ч аваагүй бол $s_a$-г авсан хүүхдийг $g_1$-ээр солиход тоо буурахгүй. Аль ч тохиолдолд $g_1$–$s_a$ хостой оновчтой шийд оршин байна. Үлдсэн хэсэгт индукцээр. $s_a$-аас жижиг жигнэмгүүд $g_1$-ийг хангахгүй тул илүү их шуналтай хэнийг ч хангахгүй, хаяхад алдагдалгүй.

## 4.3 Хэрэгжүүлэлт

**Python** (`p3_assign_cookies.py`): `list.sort` (Timsort). **Java** (`AssignCookies.java`): `Arrays.sort` (`int[]`-д dual-pivot quicksort). Хоёулаа массивыг байрандаа эрэмбэлдэг.

## 4.4 Тестийн үр дүн

LeetCode-ийн жишээ ($[1,2,3], [1,1] \to 1$; $[1,2], [1,2,3] \to 2$) болон 0–8 хүүхэд, 0–8 жигнэмэгтэй 500 санамсаргүй тохиолдолд хоёр талт графын хамгийн их хослолыг Куны аргаар олсон хариутай таарсан.

## 4.5 Хугацаа ба санах ой

Эрэмбэлэлт $\Theta(n \lg n + m \lg m)$, хоёр заагчийн хэсэг $\Theta(n + m)$. Нэмэлт санах ой нь Java-д $\Theta(\lg n)$ (quicksort-ын стек), Python-ы Timsort-д хамгийн ихдээ $n/2$ заагч.

| n = m | Хариу | Python, ms | Java, ms | Python оргил, KB | Java хуваарилсан, KB |
|---|---|---|---|---|---|
| 10 000 | 9902 | 12.9 | 3.442 | 38.2 | 0.0 |
| 100 000 | 99965 | 128.0 | 20.6 | 389.3 | 0.0 |
| 1 000 000 | 999841 | 1 172 | 256.5 | 3 844 | 0.0 |

n 10 дахин өсөхөд хугацаа Python-д 9–10, Java-д 6–12 дахин өсөж, n lg n-тэй нийцэж байна. Java-д хуваарилалт 0 (байрандаа эрэмбэлдэг). Python-ы оргил нь Timsort-ын түр массив ($n = 10^6$: 3.8 MB $\approx n/2$ заагч $\times$ 8 байт).

# 5 Бодлого 4. Jump Game II (LeetCode 45)

## 5.1 Зорилго

nums[i] нь i байрлалаас урагш хамгийн ихдээ хэдэн алхам үсэрч болохыг заана. 0-ээс n − 1-д хүрэх хамгийн цөөн үсрэлтийн тоог олно (хүрэх боломжтой гэж баталгаатай).

## 5.2 Алгоритмын санаа (B төвшин): динамик программчлал

dp[i] нь 0-ээс i-д хүрэх хамгийн цөөн үсрэлт. dp[0] = 0, i бүрээс хүрч болох j = i + 1, . . . , i + nums[i] байрлалыг dp[j] = min(dp[j], dp[i] + 1) гэж шинэчилнэ.

```
def jump(nums):
    n = len(nums)
    dp = [n] * n
    dp[0] = 0
    for i in range(n - 1):
        for j in range(i + 1, min(n, i + nums[i] + 1)):
            dp[j] = min(dp[j], dp[i] + 1)
    return dp[-1]
```

**Хугацаа ба санах ой.** Шинэчлэлийн тоо $\Sigma$ min(nums[i], n − 1 − i) буюу $\mathbf{O(n \cdot \max nums)}$, хамгийн муу үед $\mathbf{\Theta(n^2)}$ (nums[i] $\geq$ n үед яг n(n − 1)/2, тестээр шалгав). Санах ой $\Theta$(n).

## 5.3 Сайжруулсан шийдэл (A төвшин): хомхойлох арга = далд гүехэн хайлт

Байрлалуудыг орой, i-ээс i + 1, . . . , i + nums[i] уруу ирмэг гэж үзвэл бодлого нь 0-ээс n − 1 хүртэлх **хамгийн богино зам**, өөрөөр хэлбэл BFS. Гол ажиглалт: k үсрэлтээр хүрч болох байрлалууд нь **тасралтгүй муж** $[\mathrm{lo}_k, \mathrm{hi}_k]$ (i-д хүрч чадвал түүнээс өмнөх бүх байрлалд хүрч чадна). Иймд BFS-ийн дараалал хадгалах шаардлагагүй, хоёр хил хангалттай:

$$\mathrm{hi}_{k+1} = \max\{\, i + \mathrm{nums}[i] : i \leq \mathrm{hi}_k \,\}.$$

Тиймээс төвшин бүрээс «хамгийн хол хүргэх» байрлалыг сонгох нь хомхойлох сонголт бөгөөд түүний зөв байдал нь BFS-ийн төвшний зөв байдлаас шууд гарна. Хомхойлох сонголтыг ашиглах болсон үндэслэл: бүх байрлалын хамгийн цөөн үсрэлтийг (dp) мэдэх шаардлагагүй, зөвхөн төвшний хил чухал.

```
def jump(nums):
    jumps = current_end = farthest = 0
    for i in range(len(nums) - 1):
        farthest = max(farthest, i + nums[i])
```

```
        if i == current_end:          # k-р төвшин дууслаа
            jumps += 1
            current_end = farthest    # (k+1)-р төвшний хил
            if current_end >= len(nums) - 1:
                break
    return jumps
```

Массивыг нэг гүйнэ: $\Theta(\mathbf{n})$ хугацаа, $\Theta(\mathbf{1})$ санах ой.

### 5.4 Хэрэгжүүлэлт ба тестийн үр дүн

- Анхны: `p4_jump_game.py`, `JumpGame.java`.
- Сайжруулсан: `p4_jump_game_improved.py`, `JumpGameImproved.java`.

Анхны хувилбар `steps` тоолуураар dp-гийн шинэчлэлийн тоог бүртгэнэ. LeetCode-ийн жишээ ([2, 3, 1, 1, 4] → 2, [2, 3, 0, 1, 4] → 2), [0] → 0 болон 1–40 урттай 500 санамсаргүй массив дээр ДП, хомхойлох арга, тусдаа бичсэн BFS гурвын хариу ижил.

### 5.5 Харьцуулалт

nums[i] ∈ [1, 1000] санамсаргүй (LeetCode-ийн хязгаар). Python-д $n = 10^5$ үед ДП $\approx 5{\cdot}10^7$ шинэчлэл ($\approx 6$ с) хийх тул хэмжээгүй.

| | | | Python, ms | | Java, ms | | Python, KB | |
|---|---|---|---|---|---|---|---|---|
| n | Үсрэлт | ДП-гийн алхам | ДП | хомхойлох | ДП | хомхойлох | ДП | хомхойлох |
| 1 000 | 2 | 330 388 | 41.7 | 0.0447 | 2.787 | 0.0576 | 8.2 | 0.2 |
| 10 000 | 11 | 4 837 907 | 582.4 | 0.771 | 7.085 | 0.142 | 78.5 | 0.2 |
| 100 000 | 105 | 49 985 117 | — | 8.571 | 103.6 | 0.164 | — | 0.2 |

**Дүгнэлт.** ДП-гийн ажил n·(дундаж nums) ≈ 500n бол хомхойлох аргынх яг $n - 1$ алхам. $n = 10^4$ үед Python-д **755 дахин**, Java-д $n = 10^5$ үед **632 дахин** хурдан. Санах ой $\Theta(n)$ (Java-д 4n байт)-ээс $\Theta(1)$ болсон.

## 6 Бодлого 5. Open the Lock (LeetCode 752)

### 6.1 Зорилго

4 дугуйтай цоож «0000»-оос эхэлнэ. Нэг алхамд нэг дугуйг нэгээр эргүүлнэ ($9 \to 0, 0 \to 9$). Түгжих хослол (deadends)-д орж болохгүй. Зорилгод хүрэх хамгийн цөөн алхмыг олно, хүрэх боломжгүй бол $-1$.

### 6.2 Алгоритмын санаа (B төвшин): гүехэн хайлт

Төлөв бүр ($10^4$ ширхэг) нь орой, нэг алхмаар шилжих 8 төлөв нь хөрш. Ирмэгүүд жингүй тул **BFS** анх хүрсэн төвшин нь хамгийн цөөн алхам болно. Түгжих хослол уруу орохгүй, үзсэн төлөвийг дахин оруулахгүй.

```
def openLock(deadends, target):
    dead = set(deadends)
    if "0000" in dead: return -1
    if target == "0000": return 0
    seen, queue = {"0000"}, deque([("0000", 0)])
```

```
    while queue:
        state, dist = queue.popleft()
        for i in range(4):
            for d in (-1, 1):
                nxt = state[:i] + str((int(state[i]) + d) % 10) + state[i + 1:]
                if nxt in seen or nxt in dead:
                    continue
                if nxt == target:
                    return dist + 1
                seen.add(nxt)
                queue.append((nxt, dist + 1))
    return -1
```

**Хугацаа ба санах ой.** Хамгийн ихдээ $10^4$ төлөв, тус бүрд 8 хөрш, хөрш бүрд 4 тэмдэгттэй мөр үүсгэж, толинд хайна: $O(10^4 \cdot 8 \cdot 4)$. Санах ой $\Theta(10^4)$ (seen, дараалал).

### 6.3 Сайжруулсан шийдэл (A төвшин): хоёр талын гүехэн хайлт

1. **Төлөвийг бүхэл тоогоор кодлох.** «0000»…«9999» нь 0…9999. Хөршийг 10-ын зэрэг $p \in \{1000, 100, 10, 1\}$-ийг нэмж, хасаж олно (орны цифр 9 бол $+1$ нь $-9p$, 0 бол $-1$ нь $+9p$). Мөр зүсэх, шинэ мөр үүсгэх, толь хайх зардал алга. Үзсэн ба түгжих төлөвийг $10^4$ урттай байт массивт хадгална.
2. **Хоёр талаас хайх.** «0000» ба зорилгоос ээлжлэн BFS хийж, алхам бүрд **жижиг** хилийг нь нэг төвшин тэлнэ. Нэг талын хайлт нөгөө талын хүрсэн төлөвтэй уулзмагц хариу нь хоёр талын төвшний нийлбэр.

Салбарлалт b (энд 8), хариу d бол нэг талын BFS ойролцоогоор $b^d$ төлөв, хоёр талынх $2b^{d/2}$ төлөв нээнэ. Мөн аль нэг тал түгжээгээр бүслэгдсэн бол тэр талын хил шууд хоосорч, хариу $-1$ гэдэг нь бараг тэр дороо тодорхой болно.

**Зөв байдал.** Уулзсан у төлөвийг s тал $d_s$ төвшинд, нөгөө тал $t \le d_o$ төвшинд хүрсэн гэе. Хэрэв $t < d_o$ бол нөгөө тал $(t+1)$-р төвшнийг тэлэхдээ у-ийн хөрш х-д аль хэдийн хүрч, уулзалтыг эрт илрүүлэх байсан. Иймд анхны уулзалт хамгийн богино замыг өгнө. 300 санамсаргүй тохиолдолд энгийн BFS-тэй тулгаж батлав.

### 6.4 Хэрэгжүүлэлт ба тестийн үр дүн

- Анхны: `p5_open_lock.py`, `OpenLock.java` (Java-д төвшин тус бүрээр нь BFS, `HashSet<String>`).
- Сайжруулсан: `p5_open_lock_improved.py`, `OpenLockImproved.java`.

Хоёулаа `expanded` тоолуураар тэлсэн төлөвийг бүртгэнэ. LeetCode-ийн 3 жишээ $(6, 1, -1)$, «0000» түгжигдсэн, зорилго өөрөө түгжих хослол гэсэн тохиолдлууд болон 300 санамсаргүй тохиолдлыг давсан.

### 6.5 Харьцуулалт

| | | Тэлсэн төлөв | | Python, ms | | Java, ms | | Python, KB | |
|---|---|---|---|---|---|---|---|---|---|
| **Тохиолдол** | **Хариу** | **BFS** | **хоёр тал** | **BFS** | **хоёр тал** | **BFS** | **хоёр тал** | **BFS** | **хоёр тал** |
| «5555», түгжээгүй | 20 | 9 992 | 7 491 | 47.2 | 12.3 | 18.2 | 1.339 | 1 033 | 151.7 |
| LeetCode жишээ 1 | 6 | 417 | 46 | 2.013 | 0.0848 | 1.355 | 0.264 | 78.0 | 24.2 |
| 500 санамсаргүй түгжээ | 8 | 1 282 | 163 | 5.432 | 0.312 | 2.232 | 0.125 | 270.7 | 31.9 |
| зорилгыг бүсэлсэн | -1 | 9 991 | 2 | 49.6 | 0.0078 | 8.630 | 0.0190 | 1 033 | 20.2 |

**Дүгнэлт.**

- **LeetCode-ийн жишээ, санамсаргүй түгжээ:** тэлсэн төлөв 8–9 дахин цөөрч, хугацаа Python-д 17–24, Java-д 5–18 дахин буурсан.

- **«5555» (хамгийн хол, 20 алхам):** хариу бүх төлөвийн «төгсгөлд» байрладаг тул хоёр тал ч ихэнх төлөвийг нээнэ (9992 → 7491). Хурдсалт нь голчлон бүхэл тоон кодлолоос гарсан (Python 3.8, Java 13.6 дахин).
- **Бүслэгдсэн зорилго:** энгийн BFS бүх 9991 төлөвийг нээж байж −1 гэж мэддэг бол хоёр талын хайлт зорилгын талаас 1 алхам хийгээд хил хоосорсныг мэднэ (2 төлөв). Python-д **6 000 гаруй дахин** хурдан.
- **Санах ой:** Python-д 1 MB-аас 0.15 MB, Java-д 4.4 MB-аас 0.1 MB болж буурсан. String объект үүсгэхгүй, байт массив ашигласантай холбоотой.

# 7 Дүгнэлт

| № | Бодлого, арга | Хугацаа | Нэмэлт санах ой |
|---|---|---|---|
| 1 | strStr — энгийн | $O(n \cdot m)$ | $\Theta(1)$ |
| 1 | strStr — KMP | $\Theta(n + m)$ | $\Theta(m)$ |
| 2 | Flood Fill — BFS | $\Theta(R \cdot C)$ | $O(R \cdot C)$ |
| 3 | Assign Cookies — хомхойлох | $\Theta(n \lg n + m \lg m)$ | $\Theta(\lg n) \ldots \Theta(n)$ |
| 4 | Jump Game II — ДП | $O(n \cdot \max \text{nums}) \leq \Theta(n^2)$ | $\Theta(n)$ |
| 4 | Jump Game II — хомхойлох (далд BFS) | $\Theta(n)$ | $\Theta(1)$ |
| 5 | Open the Lock — BFS | $O(10^4 \cdot 8 \cdot 4)$ | $\Theta(10^4)$ |
| 5 | Open the Lock — хоёр талын BFS | $O(2b^{d/2})$ төлөв | $\Theta(10^4)$ байт |

**Гол дүгнэлтүүд:**

1. **Хомхойлох арга нь бүтцийн ажиглалт дээр тулгуурладаг.** Jump Game II-д «k үсрэлтээр хүрэх байрлалууд тасралтгүй муж» гэсэн ажиглалт BFS-ийн дараалыг хоёр хувьсагч болгож, $\Theta(n^2)$ ДП-г $\Theta(n)$ болгосон. Assign Cookies-ийн хомхойлох санголтыг солих аргаар баталсан.
2. **Хамгийн муу тохиолдлын баталгаа чухал.** KMP санамсаргүй текстэд энгийн аргаас хурдан биш боловч муу оролтод 39 дахин хурдан.
3. **Гүехэн хайлтын хурд нь нээх төлөвийн тоо ба нэг төлөвийн өртгөөс хамаарна.** Open the Lock-д хоёр талын хайлт төлөвийн тоог, бүхэл тоон кодлол нэг төлөвийн өртгийг багасгасан.
4. **Санамсаргүй тулгалт алдааг илрүүлдэг.** Энгийн BFS-ийн «зорилго түгжигдсэн» алдааг сайжруулсан хувилбартай тулгах тест илрүүлсэн.
5. **Python, Java хоёрын хандлага ижил, тогтмол үржүүлэгч өөр.** Java 2.6–52 дахин хурдан байсан ч хэмжээ өсөхөд хугацаа өсөх хурд хоёр хэлэнд ижил байв.

# A Java хэрэгжүүлэлт

Python кодыг бодлого бүрийн «Алгоритмын санаа» хэсэгт үзүүлсэн. Доор Java шийдлүүдийг `java/src/main/java/com/prj2/` хавтаснаас бүтнээр нь оруулав.

## A.1 1. strStr (`StrStr.java`)

```
package com.prj2;

/** LeetCode 28. KMP: Θ(n + m) хугацаа, Θ(m) санах ой. String.indexOf/contains ашиглахгүй. */
public class StrStr {

    public int strStr(String haystack, String needle) {
        int m = needle.length();
        if (m == 0) return 0;
        int[] fail = new int[m]; // needle[0..i]-ийн хамгийн урт жинхэнэ угтвар = дагавар
        for (int i = 1, k = 0; i < m; i++) {
            while (k > 0 && needle.charAt(i) != needle.charAt(k)) k = fail[k - 1];
            if (needle.charAt(i) == needle.charAt(k)) k++;
            fail[i] = k;
        }
        for (int i = 0, k = 0; i < haystack.length(); i++) {
            char ch = haystack.charAt(i);
            while (k > 0 && ch != needle.charAt(k)) k = fail[k - 1];
            if (ch == needle.charAt(k) && ++k == m) return i - m + 1;
        }
        return -1;
    }

    /** Харьцуулах: байрлал бүрээс тэмдэгт тэмдэгтээр, O(n·m). */
    public int strStrNaive(String haystack, String needle) {
        int n = haystack.length();
        int m = needle.length();
        for (int start = 0; start + m <= n; start++) {
            int j = 0;
            while (j < m && haystack.charAt(start + j) == needle.charAt(j)) j++;
            if (j == m) return start;
        }
        return -1;
    }
}
```

## A.2 2. Flood Fill (`FloodFill.java`)

```
package com.prj2;

/**
 * LeetCode 733. Гүехэн хайлт (BFS): нүдийг дараалалд оруулахдаа өнгийг нь сольдог тул нүд бүр
 * нэг удаа орно. Θ(R·C) хугацаа, дараалал Θ(R·C) санах ой (нүдийг r·C + c гэж int-ээр кодолно).
 */
public class FloodFill {

    private static final int[] DR = {-1, 1, 0, 0};
    private static final int[] DC = {0, 0, -1, 1};

    public int[][] floodFill(int[][] image, int sr, int sc, int color) {
        int old = image[sr][sc];
        if (old == color) return image;
        int rows = image.length;
        int cols = image[0].length;
        int[] queue = new int[rows * cols];
```

```
        int head = 0;
        int tail = 0;
        image[sr][sc] = color;
        queue[tail++] = sr * cols + sc;
        while (head < tail) {
            int cell = queue[head++];
            int r = cell / cols;
            int c = cell % cols;
            for (int d = 0; d < 4; d++) {
                int nr = r + DR[d];
                int nc = c + DC[d];
                if (nr >= 0 && nr < rows && nc >= 0 && nc < cols && image[nr][nc] == old) {
                    image[nr][nc] = color;
                    queue[tail++] = nr * cols + nc;
                }
            }
        }
        return image;
    }
}
```

## A.3 3. Assign Cookies (AssignCookies.java)

```
package com.prj2;

import java.util.Arrays;

/**
 * LeetCode 455. Хомхойлох арга: эрэмбэлээд хамгийн бага шуналтай хүүхдэд түүнийг хангах хамгийн
 * жижиг жигнэмгийг өгнө. Θ(n lg n + m lg m) хугацаа, нэмэлт Θ(1) санах ой (байрандаа эрэмбэлнэ).
 */
public class AssignCookies {

    public int findContentChildren(int[] g, int[] s) {
        Arrays.sort(g);
        Arrays.sort(s);
        int child = 0;
        for (int cookie : s) {
            if (child < g.length && cookie >= g[child]) child++;
        }
        return child;
    }
}
```

## A.4 4. Jump Game II — анхны (JumpGame.java)

```
package com.prj2;

import java.util.Arrays;

/** LeetCode 45 (B төвшин). Динамик программчлал: dp[j] = min(dp[j], dp[i] + 1). O(n^2), Θ(n) санах ой. */
public class JumpGame {

    /** dp-г шинэчилсэн тоо (хэмжилтэд). */
    public long steps = 0;

    public int jump(int[] nums) {
        int n = nums.length;
        int[] dp = new int[n];
        Arrays.fill(dp, Integer.MAX_VALUE);
        dp[0] = 0;
```

```
        for (int i = 0; i < n - 1; i++) {
            if (dp[i] == Integer.MAX_VALUE) continue;
            int end = (int) Math.min(n - 1, (long) i + nums[i]);
            for (int j = i + 1; j <= end; j++) {
                steps++;
                if (dp[i] + 1 < dp[j]) dp[j] = dp[i] + 1;
            }
        }
        return dp[n - 1];
    }
}
```

### A.5 4. Jump Game II — сайжруулсан (JumpGameImproved.java)

```
package com.prj2;

/**
 * LeetCode 45 - сайжруулсан (A төвшин). Хомхойлох арга = далд BFS: k үсрэлтээр хүрэх байрлалууд
 * тасралтгүй муж бөгөөд төвшин дуусах бүрд хамгийн хол хүрэх цэг уруу шилжинэ. Θ(n), Θ(1) санах ой.
 */
public class JumpGameImproved {

    public int jump(int[] nums) {
        int jumps = 0;
        int currentEnd = 0;
        int farthest = 0;
        for (int i = 0; i < nums.length - 1; i++) {
            farthest = Math.max(farthest, i + nums[i]);
            if (i == currentEnd) { // энэ төвшин дууслаа
                jumps++;
                currentEnd = farthest;
                if (currentEnd >= nums.length - 1) break;
            }
        }
        return jumps;
    }
}
```

### A.6 5. Open the Lock — анхны (OpenLock.java)

```
package com.prj2;

import java.util.ArrayDeque;
import java.util.Arrays;
import java.util.HashSet;
import java.util.Set;

/**
 * LeetCode 752 (B төвшин). Гүехэн хайлт: «0000»-ооc эхэлж, төвшин бүрд 8 хөршийг (дугуй бүрийг
 * ±1) нэмнэ. Төлөвийг String-ээр, үзсэнийг HashSet-ээр хадгална. O(10^4·8·4), Θ(10^4) санах ой.
 */
public class OpenLock {

    /** Дараалаас гаргасан төлөвийн тоо (хэмжилтэд). */
    public long expanded = 0;

    public int openLock(String[] deadends, String target) {
        Set<String> dead = new HashSet<>(Arrays.asList(deadends));
        if (dead.contains("0000")) return -1;
        if (target.equals("0000")) return 0;
        Set<String> seen = new HashSet<>();
```

```
        seen.add("0000");
        ArrayDeque<String> queue = new ArrayDeque<>();
        queue.add("0000");
        for (int dist = 1; !queue.isEmpty(); dist++) {
            for (int size = queue.size(); size > 0; size--) { // нэг төвшин
                String state = queue.poll();
                expanded++;
                char[] digits = state.toCharArray();
                for (int i = 0; i < 4; i++) {
                    char original = digits[i];
                    for (int d = -1; d <= 1; d += 2) {
                        digits[i] = (char) ('0' + (original - '0' + d + 10) % 10);
                        String next = new String(digits);
                        if (seen.contains(next) || dead.contains(next)) continue;
                        if (next.equals(target)) return dist;
                        seen.add(next);
                        queue.add(next);
                    }
                    digits[i] = original;
                }
            }
        }
        return -1;
    }
}
```

## A.7 5. Open the Lock — сайжруулсан (OpenLockImproved.java)

```
package com.prj2;

/**
 * LeetCode 752 - сайжруулсан (A төвшин): хоёр талын гүехэн хайлт, төлөв нь 0...9999 бүхэл тоо.
 * <ul>
 *   <li>Хөршийг 10-ын зэргээр нэмж/хасаж олно - String үүсгэхгүй; side[] нь byte массив.</li>
 *   <li>«0000» ба зорилгоос ээлжлэн, жижиг хилийг нь нэг төвшин тэлнэ. Хоёр хил уулзмагц
 *       хариу = хоёр талын төвшний нийлбэр. Нэг талын BFS ≈ b^d, хоёр талынх ≈ 2·b^(d/2) төлөв.</li>
 * </ul>
 */
public class OpenLockImproved {

    private static final int[] POW = {1000, 100, 10, 1};

    /** Тэлсэн төлөвийн тоо (хэмжилтэд). */
    public long expanded = 0;

    public int openLock(String[] deadends, String target) {
        byte[] dead = new byte[10000];
        for (String d : deadends) dead[Integer.parseInt(d)] = 1;
        int goal = Integer.parseInt(target);
        if (dead[0] == 1 || dead[goal] == 1) return -1;
        if (goal == 0) return 0;
        byte[] side = new byte[10000]; // 0 - үзээгүй, 1 - эхлэлээс, 2 - зорилгоос
        side[0] = 1;
        side[goal] = 2;
        int[][] front = {null, {0}, {goal}};
        int[] size = {0, 1, 1};
        int[] depth = {0, 0, 0};
        int[] buffer = new int[10000];
        while (size[1] > 0 && size[2] > 0) {
            int s = size[1] <= size[2] ? 1 : 2; // жижиг хилийг тэлнэ
            int other = 3 - s;
            depth[s]++;
            int count = 0;
            for (int idx = 0; idx < size[s]; idx++) {
```

```
                int x = front[s][idx];
                expanded++;
                for (int p : POW) {
                    int digit = x / p % 10;
                    for (int k = 0; k < 2; k++) {
                        int y = k == 0 ? (digit < 9 ? x + p : x - 9 * p)  // +1 (9 → 0)
                                       : (digit > 0 ? x - p : x + 9 * p); // −1 (0 → 9)
                        if (dead[y] == 1 || side[y] == s) continue;
                        if (side[y] == other) return depth[1] + depth[2];
                        side[y] = (byte) s;
                        buffer[count++] = y;
                    }
                }
            }
            front[s] = java.util.Arrays.copyOf(buffer, count);
            size[s] = count;
        }
        return -1;
    }
}
```